PRICING AZERBAIJANI REAL ESTATE FROM SCRATCH

A NumPy CART decision tree and a Pegasos SVM on bina.az property listings

Samur Eyyubov · Ali Huseynli · Polad Ibrahimli · Tural Usubov

AI Academy — National Artificial Intelligence Center

ML-502 Machine Learning · Final Project · October 2026

`github.com/eyyubovsamur240-afk/ML-final-project-main`

PROBLEM & DATA

One Real-World Dataset, Two Machine Learning Tasks

The question

Can models we write ourselves learn how bina.az asking prices depend on size, location, property category, floor and repair state?

Why the raw scrape is hard

- **Repeated scrapes:** up to 17 copies of one listing
- **Units inside text:** “145 m²”, “1.3 sot”, floor “7 / 9”
- **Missing by design:** no floor for land, no rooms for garages
- **Entry errors:** from an 11 AZN unit to a 600 M AZN flat
- **Azerbaijani categoricals:** 142 districts, 7 categories
- **Location matters:** 99.6% of listings are in Baku

TASK A · REGRESSION

metric: RMSE on log(price)

Listing attributes
115 encoded features

→

Our CART regressor
`decision_tree.py`

→

Asking price
scored on log(price)

TASK B · CLASSIFICATION

metrics: F1 (premium), ROC-AUC

Listing attributes
115 encoded features

→

Our CART tree

Our Pegasos SVM

→

Premium vs standard
price > 209,000 AZN
= training-split median

100,775
raw rows · 51 columns

63,463
unique listings

115
features

70 / 15 / 15
44,423 / 9,520 / 9,520 rows · seed 42

Eyyubov · Huseynli · Ibrahimli · Usubov
AI Academy · National AI Center

DATA PIPELINE

From 100,775 Scraped Rows to 63,463 Valid Listings

Cleaning ledger · src/data_prep.clean_data

step	rows left	removed
Raw Kaggle dump	100,775	
Repeated scrapes	64,454	-36,321
Outlier rules	64,373	-81
Same ID, other URL	63,704	-669
Exact re-posts	63,651	-53
Unknown room count	63,464	-187
Coords outside AZ	63,463	-1

1. **Keep the newest scrape of each URL:** 3,356 listings changed price between scrapes
2. **Price per m² only flags errors** — it is never a model input
3. **Fixed rules per category, not learned quantiles:** 81 rows (0.13%)

src/data_prep.py — remove_repeated_scrapes · apply_outlier_rules

```
59 def remove_repeated_scrapes(df: pd.DataFrame) -> pd.DataFrame:
61     df = df.copy()
62     # Timestamps come in mixed formats (some with '+00', some without)
63     df["scraped_at"] = pd.to_datetime(
           df["datetime_scrape_y"], format="mixed", utc=True)
1  64     return (df.sort_values(["estate_rel_url_x", "scraped_at"])
                 .drop_duplicates(subset="estate_rel_url_x", keep="last")
                 .reset_index(drop=True))
```

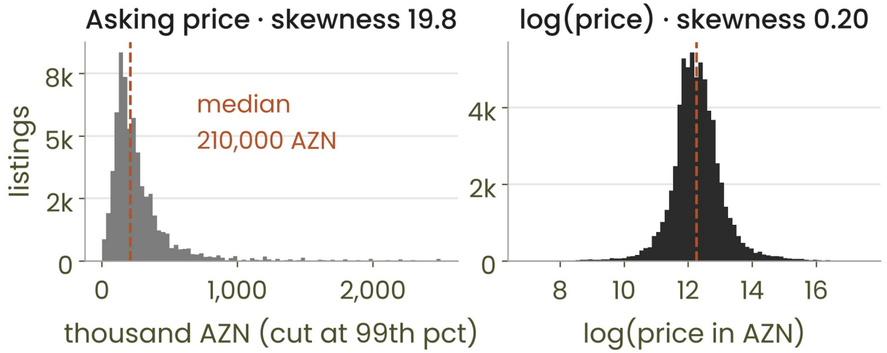
apply_outlier_rules — excerpt (lines 79–93)

```
79     has_building = df["area_m2"] > 0
80     is_land = category == LAND
2  83     price_per_m2 = df["price"] / df["area_m2"].where(has_building)
87     rules = {
88         "price < 5,000 AZN (buildings)":
3  89             has_building & (df["price"] < 5_000),
90         "residential price < 15,000 AZN":
91             is_residential & (df["price"] < 15_000),
92         "residential area < 15 m2":
93             is_residential & (df["area_m2"] < 15),
```

EDA & FEATURES

Understanding the Market Before Training

Asking price is heavy-tailed; log(price) is the training scale



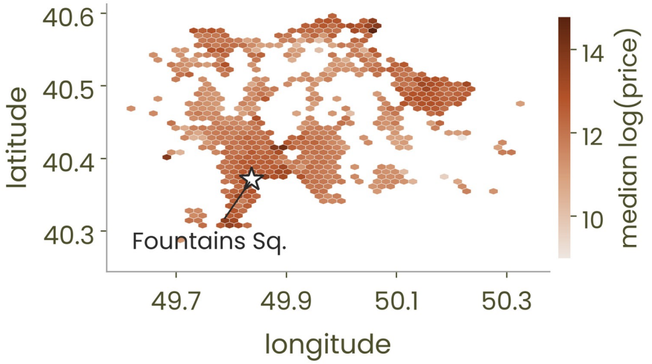
Spearman ρ with price

+0.74
floor area

+0.52
number of rooms

-0.40
km to city centre

Location: median log(price) per hexagon, Baku



Re-plotted for projection from the clean table of src/data_prep.clean_data (same binning as src/eda.py, report Figs. 1 and 3).

Engineered from raw text

- Sahə → floor area (m²) or land area (sot)
- "7 / 9" → floor and total floors
- missing rooms recovered from "4 otaqlı"
- haversine km to Fountains Square
- one-hot category, repair, district → 115 columns

Leakage removed (price task)

- **unit_price, total_price**: functions of the target, dropped and asserted absent
- identifiers: owner, shop, address, URLs
- views: known only after the listing is live
- price per m²: outlier flag and Task A target only

Learned from training rows only

- **tier threshold 209,000 AZN** (training median)
- districts kept if ≥ 30 training listings (92 of 142)
- one-hot layout; unseen category → all zeros
- SVM scaling: mean/std of each fit's own rows

DECISION TREE

Implementing CART from Scratch in NumPy

src/decision_tree.py — DecisionTree._best_split — vectorised split search

```
189 def _best_split(self, X, y):
208     # sort every feature column at once; a Python loop over features
210     order = np.argsort(Xf, axis=0, kind="stable")
211     x_sorted = np.take_along_axis(Xf, order, axis=0)
212     scores = self._split_scores(y[order])           # (n-1, d)
217     valid = x_sorted[1:] > x_sorted[:-1] + FEATURE_THRESHOLD
218     n_left = np.arange(1, n)
219     valid &= ((n_left >= self.min_samples_leaf)
                  & (n - n_left >= self.min_samples_leaf))[:, None]
222     scores = np.where(valid, scores, np.inf)
226     col, row = divmod(int(np.argmin(scores.T)), n - 1)
228     lo, hi = x_sorted[row, col], x_sorted[row + 1, col]
229     threshold = lo / 2.0 + hi / 2.0
235     decrease = self._impurity(y) - float(scores[row, col])
236     return int(features[col]), float(threshold), decrease
```

1. **Sort every column once:** O(n·d·log n) per node
2. **Prefix sums** → all (n-1)×d split scores
3. **Valid:** value changes, both children ≥ min leaf
4. **argmin on scores.T:** deterministic ties

Impurity and information gain

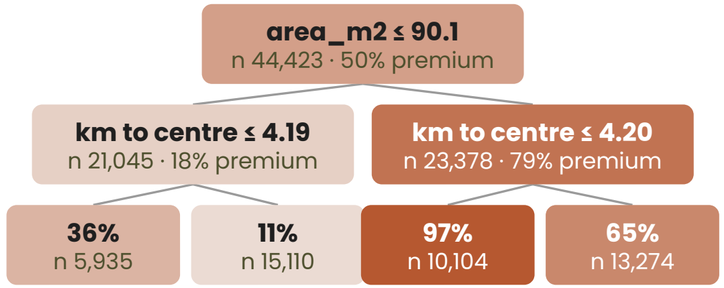
$$G = 1 - \sum_k p_k^2 \qquad H = -\sum_k p_k \log_2 p_k$$

$$I_{\text{reg}} = \frac{1}{N}\sum_i (y_i - \bar{y})^2$$

$$\text{IG} = I(\text{parent}) - \sum_c \frac{N_c}{N} I(\text{child}_c)$$

split only if $\frac{N_t}{N}\,\text{IG} \geq$ `min_impurity_decrease`

Top of our final Task B tree (entropy)



Our class re-fitted on the training rows; greedy growth keeps these top splits in the depth-20 model. Left = yes.

PEGASOS SVM

Training a Support Vector Machine Without a Library Solver

src/svm.py — PegasosSVM.fit — the whole solver

```
67  X_aug=np.hstack([X,np.ones((n_samples,1))])
68  y_pm,self.classes_=self._to_pm1(y)
69  rng=np.random.default_rng(self.random_state)
70  w_aug=np.zeros(n_features+1)
71  self.history_=[]
73  for t in range(1,self.n_iters+1):
1   74      learning_rate=1/(self.lambda_*t)   step size 1/(λt)
2   75      i=rng.integers(n_samples)   one random row per step
76      x_i=X_aug[i]
77      y_i=y_pm[i]
79      score=w_aug @ x_i
80      margin=y_i*score
3   81      if margin<1:   margin violated?
4   82          w_aug=(1-learning_rate*self.lambda_)*w_aug+learning_rate*y_i*x_i
83      else:
84          w_aug=(1-learning_rate*self.lambda_)*w_aug   shrink only
86      if t%100==0:
87          self.w,self.b=w_aug[:-1],w_aug[-1]
5   88          self.history_.append(self._objective(X,y_pm))   full-data J
```

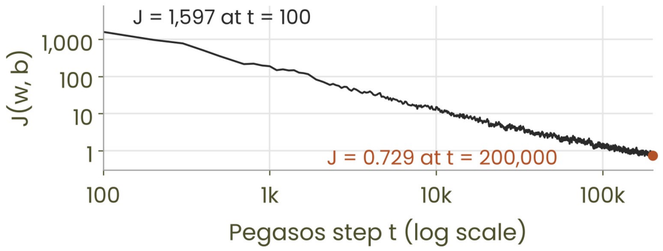
Regularised hinge-loss primal

labels mapped to ±1, bias folded into w

$$J(\mathbf{w}, b) = \frac{\lambda}{2}\|\mathbf{w}\|^2 + \frac{1}{n}\sum_{i=1}^{n}\max\left(0,\ 1 - y_i(\mathbf{w}^\top \mathbf{x}_i + b)\right)$$

Objective decreases

same settings re-run: test F1 0.867 reproduced



$\lambda = 10^{-4}$ · 200,000 steps (≈ 4.5 epochs) · seed 42

Not done: λ sweep · margin, support vectors · kernel / RFF

EXPERIMENT DESIGN

A Reproducible and Leakage-Free Model Comparison

1 Clean
100,775 → 63,463 rows

2 Split 70/15/15
seed 42, stratified

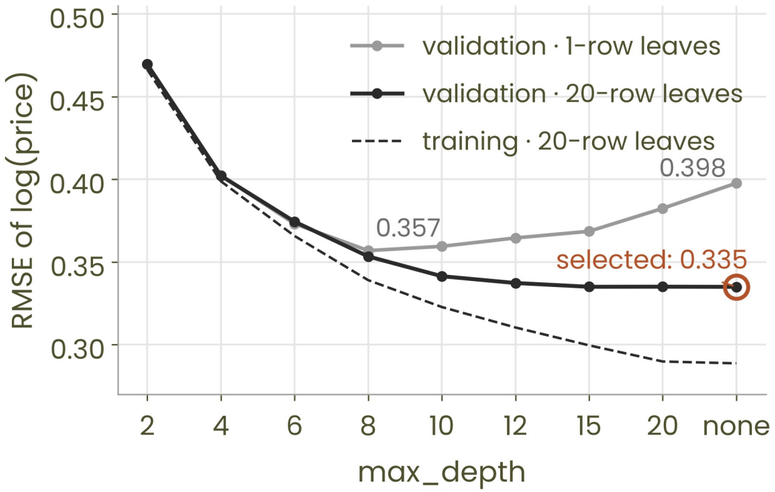
3 Preprocess
fitted on train only

4 Fit models
ours + sklearn twins

5 Tune
on validation: 9 × 5 grid

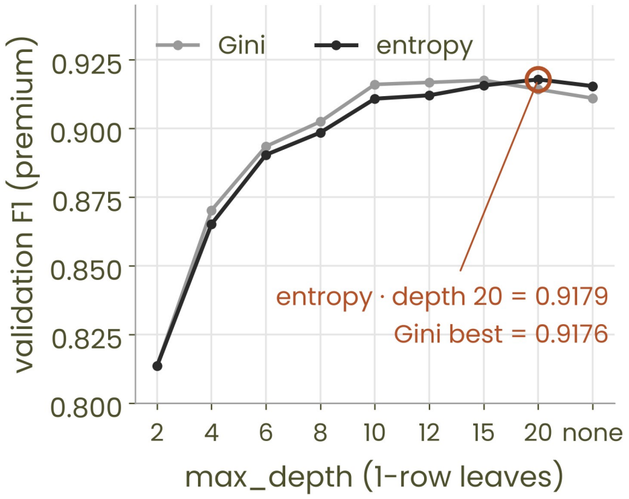
6 Test once
+ CV and bootstrap CI

Task A · depth and leaf size tuned together



validation split · source: metrics.json tree_study.task_a

Task B · Gini vs entropy



difference ≈ 0.0003 F1: criterion barely matters

Chosen on validation only

	Task A	Task B
`criterion`	**MSE**	**entropy**
`max_depth`	**none**	**20**
`min_samples_leaf`	**20**	**1**
`validation`	**RMSE 0.335**	**F1 0.918**

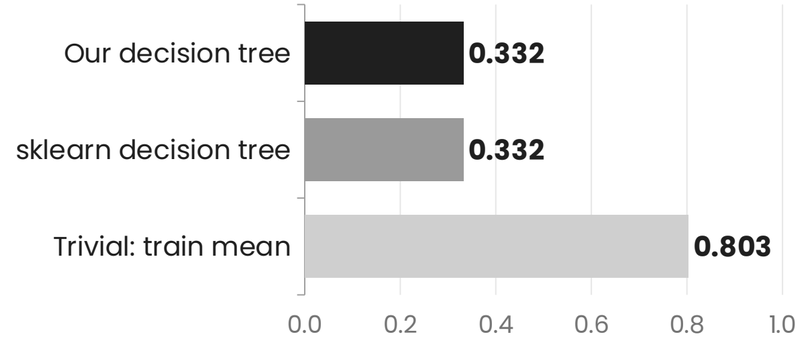
- **Task A target:** log(price / area); log(area) is added back, so every metric is on log(price)
- **SVM fixed, not tuned:** $\lambda = 10^{-4}$, 200,000 steps
- **sklearn twins:** same settings and rows; SGD $\alpha = \lambda$, LinearSVC C = 1/(λn)

Eyyubov · Huseynli · Ibrahimli · Usubov
AI Academy · National AI Center

RESULTS

Our NumPy Models Match Strong Reference Implementations

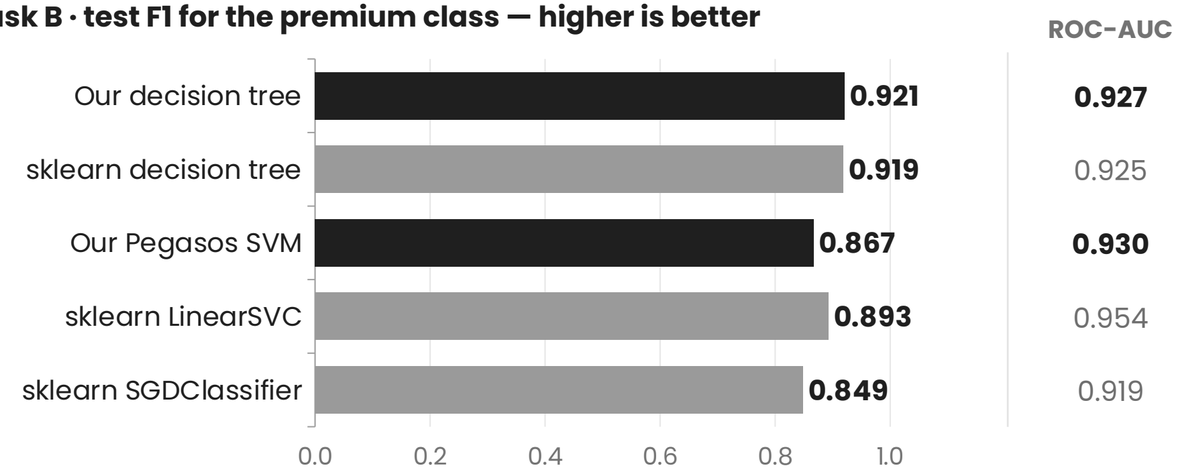
Task A · test RMSE on log(price) — lower is better



R^2 0.829 · MAE 0.185 · median APE 10.3% in AZN

95% CI: ours and sklearn [0.317, 0.349]; trivial [0.784, 0.824]
5-fold CV RMSE 0.334 ± 0.013 on the training split

Task B · test F1 for the premium class — higher is better



Tree F1 95% intervals overlap: ours [0.916, 0.927], sklearn [0.913, 0.925] (1,000 bootstrap resamples of the test split)

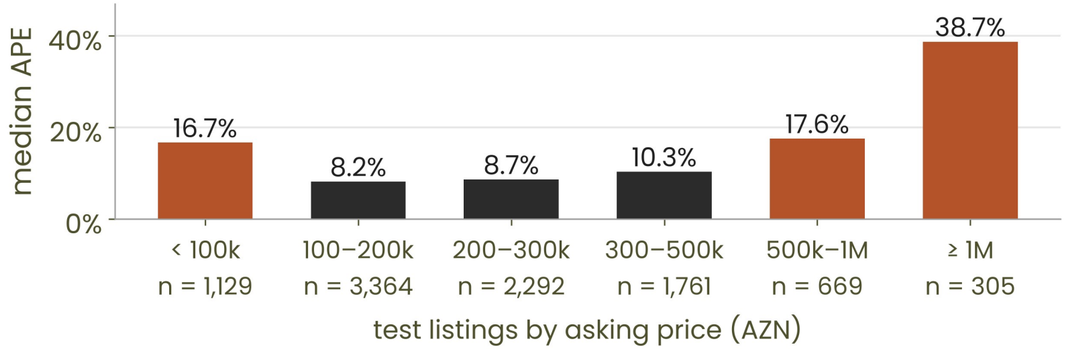
Our CART reproduces scikit-learn's accuracy; our Pegasos SVM beats SGDClassifier on the same objective and budget but stops short of the converged LinearSVC.

Same settings, same rows: 99.2% / 98.4% identical test predictions (A / B) · depth 29 / 20 · leaves 1,724 vs 1,724 and 2,690 vs 2,685 · not identical node by node (tie order; sklearn uses float32) · cost: 13–40× slower fit

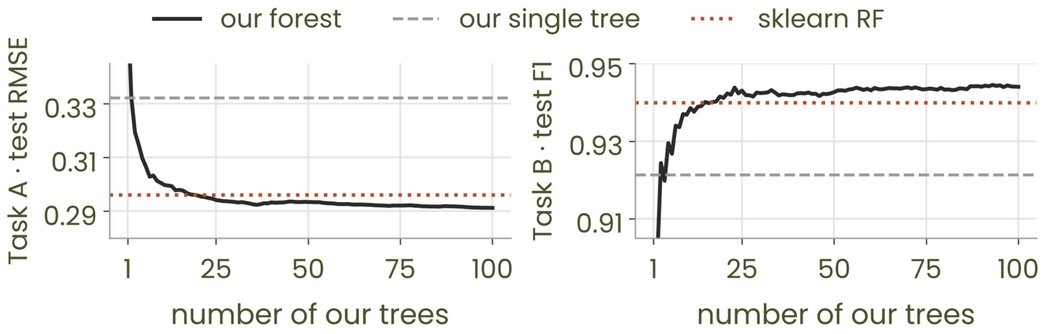
DIAGNOSTICS

What the Models Learn – and Where They Fail

Task A tree · median APE by price band (test)



Bonus · forest of 100 of our trees: RMSE 0.332 → 0.291 · F1 0.921 → 0.944



Where the errors concentrate

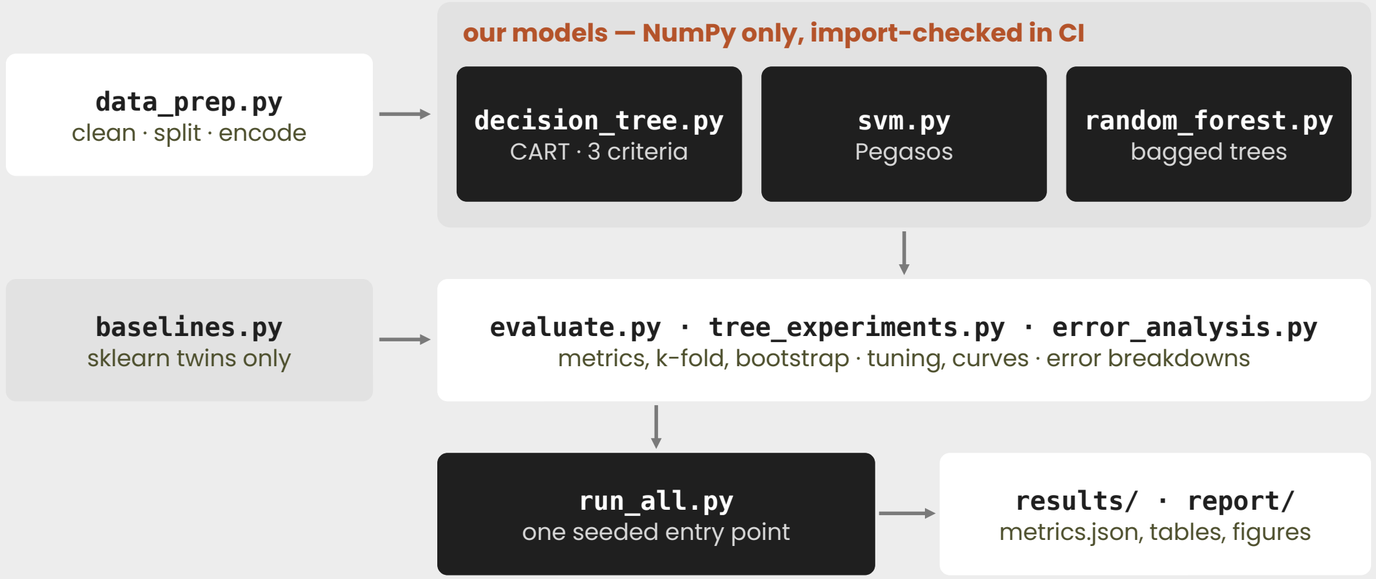
- **Drivers:** floor area, km to centre, land area (permutation importance)
- **By type:** flats 8.1% · houses 21% · commercial 37% · land 40%
- **Sparse districts** (< 30 train listings): 26% vs 9.5%
- **Near the threshold** (± 5%): tree 37%, SVM 49% wrong

src/random_forest.py — **1** bootstrap rows **2** our DecisionTree, max_features per node **3** average

```
86  for _ in range(self.n_estimators):
1   87      rows = rng.integers(0, n, size=n) if self.bootstrap else np.arange(n)
90          seed = int(rng.integers(2**31 - 1))
2   91      tree = DecisionTree(**params, random_state=seed).fit(X[rows], y[rows])
92          self.trees.append(tree)
122 def predict(self, X):
3   123     return self._to_prediction(self._tree_outputs(X).mean(axis=0))
```

ENGINEERING

Built from First Principles, Tested Against Reality



$ python -m src.run_all

- seed 42 everywhere; a test checks two runs give identical numbers apart from timings
- 100 pytest tests: hand-computed IG = 1/3, sklearn agreement on tie-free trees, each stopping rule changes the tree

Who did what · contribution_report.md

Samur Eyyubov
evaluation layer (metrics, ROC-AUC, k-fold, bootstrap), sklearn baselines, error analysis, figures, run_all.py, CI

Ali Huseynli
data pipeline: cleaning, de-duplication, outlier and missing-value rules, leakage removal, split; EDA; tree contributions

Polad Ibrahimli
Pegasos SVM: ±1 labels, folded bias, $1/(\lambda t)$ schedule, objective history, toy-cloud validation

Tural Usubov
CART tree and vectorised split search, tuning and bias–variance study, sklearn node comparison, random forest, tests

CONCLUSIONS

What We Learned – and What Is Still Open

1 **Careful NumPy code reproduces the reference models**

Tree RMSE 0.332 = sklearn 0.332; F1 0.921 vs 0.919; 99% identical predictions. The cost is speed, not accuracy.

2 **Data preparation decides whether the numbers are credible**

36% of raw rows were repeat scrapes; target-derived columns dropped; every statistic fitted on train only.

3 **The remaining error is structural**

A linear boundary misses interactions (SVM 0.867 vs tree 0.921); land, luxury and sparse districts dominate the error.

Next steps

- **SVM:** λ and step-budget sweep, iterate averaging; report $\|w\|$, $2/\|w\|$ and support vectors
- **Non-linear SVM:** random Fourier features feeding the same Pegasos solver
- **Tails and sparse groups:** per-category models or a quantile loss; spatially blocked validation

Status against the brief

- ■ CART: Gini, entropy, MSE, 4 stopping rules, predict_proba
- ■ Pegasos SVM with objective tracking
- ■ leakage-free split, train-only preprocessing
- ■ matched sklearn benchmark, CV, bootstrap CI, timing
- ■ importances and error analysis
- ■ one-command reproduction, tests, CI
- ■ bonus: random forest of our trees

Not implemented

- ☐ SVM λ sweep (Problem 4b, partial)
- ☐ $\|w\|$, margin, support vectors (3c)
- ☐ kernel or RFF extension (3d)

THANK YOU · QUESTIONS

Reproduce every number and figure

`python -m src.run_all`

github.com/eyyubovsamur240-afk/ML-final-project-main

Backup: A · tree internals B · tests, evaluation, objective

BACKUP A

Decision Tree Internals: Prefix Sums and Stopping Rules

src/decision_tree.py — 1 prefix sums 2 total - prefix 3 weighted

```
132 n = y_sorted.shape[0]
133 n_left = np.arange(1, n)[:, None]
134 n_right = n - n_left
156 else:
160     yc = y_sorted - y_sorted.mean()
1 161     cum = np.cumsum(yc, axis=0)
162     cum_sq = np.cumsum(yc ** 2, axis=0)
163     sum_l, sq_l = cum[:-1], cum_sq[:-1]
2 164     sum_r, sq_r = cum[-1] - sum_l, cum_sq[-1] - sq_l
165     imp_left = sq_l / n_left - (sum_l / n_left) ** 2
166     imp_right = sq_r / n_right - (sum_r / n_right) ** 2
3 168 return (n_left * imp_left + n_right * imp_right) / n
```

src/decision_tree.py — _grow · stopping rules 4 depth 5 leaf size 6 weighted gain

```
255 is_leaf = (
4 256     (self.max_depth is not None and depth >= self.max_depth)
257     or n < self.min_samples_split
5 258     or n < 2 * self.min_samples_leaf
259     or node.impurity <= EPSILON         # node is already pure
260 )
261 if is_leaf:
262     return node
6 271 weighted_decrease = n / self.n_samples_fit_ * decrease
272 if weighted_decrease + EPSILON < self.min_impurity_decrease:
273     return node
```

Same arithmetic, all candidate splits at once

$$S_i = \sum_{j \le i} y_{(j)}, \quad Q_i = \sum_{j \le i} y_{(j)}^2, \quad \mathrm{Var}_L(i) = \frac{Q_i}{i} - \left(\frac{S_i}{i}\right)^2$$

$$\mathrm{Var}_R(i) = \frac{Q_n - Q_i}{n-i} - \left(\frac{S_n - S_i}{n-i}\right)^2, \qquad \mathrm{score}(i) = \frac{i\,\mathrm{Var}_L + (n-i)\,\mathrm{Var}_R}{n}$$

y is centred first so the subtraction keeps its precision on raw prices

Why a node-by-node comparison with sklearn is meaningful

- x ≤ threshold goes left; threshold = midpoint of neighbouring values that differ by > 1e-7 (sklearn's constant)
- stopping rules use sklearn's definitions; min_samples_leaf is enforced inside the split search
- ties: we take the lowest feature index, sklearn a random feature order; float32 vs float64 inputs

BACKUP B

Tests, Uncertainty and the Objective We Track

tests/test_decision_tree.py — a split checked by hand

```
73 def test_hand_computed_root_split():
74     # Root of the tiny classification tree, by hand (gini):
75     #   parent: 5 vs 5 -> G = 0.5
76     #   x1 <= 5.5: left = 6 rows (1 zero, 5 ones), right = 4 rows (all zeros)
77     #   G_left = 1 - (1/6)^2 - (5/6)^2 = 10/36, G_right = 0
78     #   IG = 0.5 - 6/10 * 10/36 = 1/3
79     tree = DecisionTree(criterion="gini")
80     tree.n_classes_ = 2
81     tree.n_features_in_ = 2
1  82     feature, threshold, decrease = tree._best_split(X_TINY, Y_TINY_CLF)
83     assert (feature, threshold) == (1, 5.5)
84     assert np.isclose(decrease, 1 / 3)
```

tools/check_golden_rule.py

```
41 PROTECTED = [
42     "src/decision_tree.py",
43     "src/svm.py",
44     "src/random_forest.py",
45 ]
50 ALLOWED_THIRD_PARTY = {"numpy"}
```

CI and pytest fail if a protected file reaches sklearn.svm / tree / linear_model / ensemble, scipy.optimize, torch …

src/evaluate.py — bootstrap_ci

```
305 rng = np.random.default_rng(seed)
306 stats = np.empty(n_boot)
307 for b in range(n_boot):
1  308     idx = rng.integers(0, n, size=n)
309     stats[b] = metric(*(a[idx] for a in arrays))
310 stats = stats[~np.isnan(stats)]
```

src/svm.py — PegasosSVM._objective

```
100 def _objective(self,X,y_pm):
101     scores=self.decision_function(X)
102     all_margins=np.multiply(y_pm,scores)
103     reg=(self.lambda_/2)*np.dot(self.w,self.w)
104     hinge=np.maximum(0,1-all_margins)
106     return reg+np.mean(hinge)
```

100 pytest tests

Re-run on NumPy 2.5: 99 pass; 1 fails only because the test calls np.trapz, removed in NumPy 2.0 (the pinned 1.26.4 has it).